# Learning a Reward Function to Train a Computer-Controlled Fighting Game Agent

Austin McGregor
Ray Wang
Jake Nardomarino
Agnibho Chattarji

## Abstract

Fighting games present a unique challenge for reinforcement learning due to their complex state spaces and real-time constraints. Standard agents often rely on hand-crafted heuristics that fail to capture high-level strategies like zoning or cornering. In this work, we propose a Deep Inverse Reinforcement Learning (IRL) approach to learn a reward function from expert gameplay data (BlackMamba) to train a Monte Carlo Tree Search (MCTS) agent (TOVOR) for the FightingICE platform. We systematically evaluate the impact of **Reward Shaping** (Sparse Outcome-based vs. Dense Physics-based) and **Model Architecture** (Fast Shallow Networks vs. Deep Networks). We demonstrate that a deep network trained on sparse, long-horizon outcome labels significantly outperforms both the baseline agent and dense-reward agents. Our best agent, **SparseDeep**, achieves a **93% win rate** against the standard MctsAi baseline, compared to the original TOVOR's 26% win rate, validating that learned strategic rewards can surpass hand-crafted heuristics.

## 1 Introduction

Fighting games are a genre of video games in which two players face against each other in simulated combat. A staple in fighting games is the existence of a computer-controlled player, or CPU. Having CPUs allows for people to play by themselves, incentivizing casual players and helping competitive players practice. Being able to increase the skill level of these CPUs will enhance these positive effects.

To promote the development of skilled CPUs, *DareFightingICE* was created as a competition where custom CPUs play against each other. Many submissions utilize reinforcement learning (RL). However, most RL approaches rely on simple, hand-crafted reward functions (e.g., maximizing HP difference). While effective, these functions are often suboptimal, potentially leading to slow convergence or "greedy" behaviors that miss long-term strategic advantages like positioning.

### 1.1 Research Question and Success Criteria

In this work, we focus on improving the **TOVOR** agent, a standard Monte Carlo Tree Search (MCTS) agent. TOVOR uses a simple heuristic that evaluates future states based strictly on the difference in health points (HP). Our goal is to replace this heuristic with a learned "brain" derived from an expert agent, **BlackMamba**, a past competition winner known for its strategic play. By applying Inverse Reinforcement Learning (IRL), we aim to teach TOVOR to value states not just by HP, but by the implicit strategic values (spacing, energy, safety) demonstrated by BlackMamba.

We compare our improved agent against the competition's standard baseline, **MctsAi**, a strong MCTS agent provided by the developers. MctsAi serves as the benchmark: if our learned reward function allows TOVOR to defeat MctsAi more consistently than the original TOVOR did, our approach is successful.

The primary objective of this research is to determine whether a learned, non-linear reward function can outperform standard hand-crafted heuristics in a real-time fighting game environment. Specifically, we investigate the impact of reward signal density on agent performance.

**Research Question:** Does a *Dense* reward signal, derived from short-term physics interactions (e.g., HP change over 10-30 frames), provide a more effective training signal for an MCTS agent than a *Sparse* reward signal based on long-term match outcomes?

**Success Criteria:** We define success quantitatively based on the **Win Rate** achieved over 100-round matches against the standard baseline agent, *MctsAi*. The project is considered successful if:

1. **Baseline Improvement:** Our best learned agent achieves a win rate statistically significantly higher than the original TOVOR agent's baseline win rate of 26%.
2. **Comparative Insight:** We can conclusively identify which combination of Reward Type (Sparse vs. Dense) and Model Architecture (Fast vs. Deep) yields the highest performance, providing insight into the trade-off between strategic foresight and tactical reactivity.

### 1.2 Significance and Implications

With a learned reward function that objectively improves the performance of computer-controlled agents, developers of CPUs benefit from faster training and more robust behaviors by moving away from brittle hand-crafted heuristics and towards generalized reward learning. Successfully demonstrating that Deep IRL can capture expert strategies (like zoning or cornering) without explicit programming would encourage broader adoption of these techniques in commercial game development.

Furthermore, our investigation into the trade-off between **Model Depth** and **Search Speed** has significant implications for real-time AI. If a shallow network with more MCTS simulations outperforms a deep network with fewer simulations, it suggests that "quantity of thought" (search depth) is more valuable than "quality of thought" (evaluation accuracy) in this domain. Conversely, if the deep network wins, it implies that strategic nuance cannot be brute-forced.

## 2 Related Work

Our approach is informed by research in both domain-specific fighting game AI and the broader literature on reinforcement learning (RL) and inverse reinforcement learning (IRL).

In the fighting game domain, Oh et al. developed pro-level AI agents for *Blade and Soul* using a deep RL approach with a reward-shaped self-play curriculum [5]. They trained agents with distinct "styles" (e.g., aggressive, defensive) by using different hand-crafted reward signals. This work demonstrates the power of reward shaping for improving learning efficiency. However, their reliance on hand-crafted rewards requires significant domain expertise and may not capture the nuanced, implicit strategies of expert players. Our work differs by seeking to automatically learn an optimized reward function from expert data via IRL, rather than designing it manually. The work on SpringAI by Kim et al. also used a hand-crafted reward based on HP difference, round results, and a time penalty, showing strong results but leaving room for improvement through learned rewards [3].

The theoretical foundation of our method is rooted in the principle of potential-based reward shaping (PBRS) [4]. PBRS provides a crucial theoretical guarantee: if a shaping reward is defined as the difference of a potential function over states, $F(s, s') = \gamma\Phi(s') - \Phi(s)$, the optimal policy of the original problem remains unchanged. While PBRS provides a framework for "safe" reward shaping, it does not specify how to design an effective potential function $\Phi$. Our project directly addresses this gap by proposing to use a deep neural network to learn this potential function from expert demonstrations.

Our approach is also situated within the broader IRL literature. Foundational surveys by Adams et al. highlight the variety of methods for inferring reward functions from expert behavior [1]. We adopt the Maximum Entropy (MaxEnt) IRL framework, which provides a principled probabilistic approach to inferring rewards from demonstrations that may be noisy or suboptimal [6]. This contrasts with more recent end-to-end frameworks like Generative Adversarial Imitation Learning (GAIL), which learn the policy and reward function simultaneously in an adversarial loop [2]. Our two-stage method—first

learning a fixed reward function, then training a policy—may offer greater training stability and allow for more direct analysis of the learned reward function itself, bridging the gap between general IRL theory and the specific challenges of the fighting game domain.

# 3 Method/Approach

## 3.1 End-to-End Pipeline

Our approach implements a complete Deep Inverse Reinforcement Learning (IRL) pipeline to train a fighting game agent. The workflow begins with raw JSON replay files generated by the FightingICE engine. We process these files using a custom Python parser that extracts 44 specific features per frame, including relative velocities, one-hot encoded states, and threat metrics. The parser assigns labels based on the source file (Expert=1, Baseline=0) and exports a structured CSV dataset. This dataset is then fed into our training script, which trains four distinct PyTorch models (SparseFast, SparseDeep, DenseFast, DenseDeep) using Binary Cross Entropy Loss. The learned weights for all four models are serialized into a single JSON file. Finally, the Java-based TOVOR agent loads these weights at runtime via a helper class. During gameplay, the agent uses Monte Carlo Tree Search to simulate future states 30 frames ahead, extracts the same 44 features from the simulated state, and queries the Neural Network to obtain a "goodness" score, which guides its decision-making process in real-time. A high-level overview of this full pipeline is shown in Fig. 1.

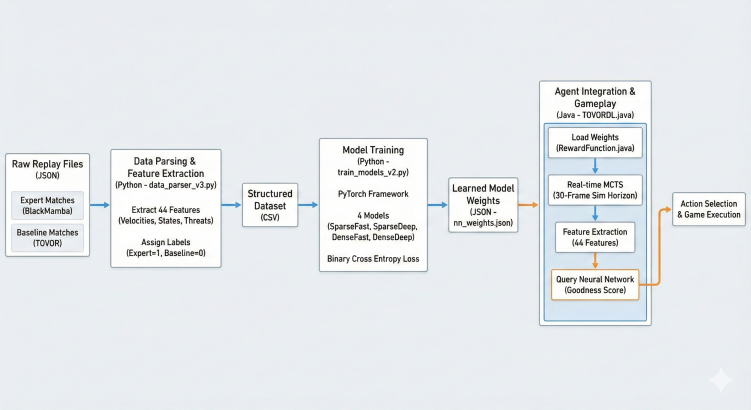


Figure 1: Overview of our end-to-end Deep Inverse Reinforcement Learning pipeline, including data parsing, feature extraction, model training, and integration into the TOVOR MCTS agent.

## 3.2 Inverse Reinforcement Learning Formulation

We frame the reward learning problem as a supervised classification task, which can be viewed as a simplified variant of Maximum Entropy Inverse Reinforcement Learning (MaxEnt IRL). In standard Reinforcement Learning, the goal is to find a policy $\pi$ that maximizes the expected cumulative reward:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{T} \gamma^t r(s_t, a_t) \right]$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory, $\gamma$ is the discount factor, and $r(s, a)$ is the reward function.

In our setting, the reward function $r(s, a)$ is unknown. We assume access to a set of expert trajectories $\mathcal{D}_E = \{\tau_E\}$ generated by the expert agent (BlackMamba), which are assumed to be optimal with respect to the unknown reward. We approximate the reward function using a parameterized neural network $R_\theta(s)$, where the input is the feature vector $\phi(s)$ extracted from the state.

Instead of solving the full iterative IRL loop (which is computationally prohibitive for real-time training), we adopt a direct classification approach. We define the probability that a state $s$ belongs to an optimal trajectory as:

$$P(\text{optimal} \mid s) = \sigma(R_\theta(\phi(s)))$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function. We effectively treat states from expert winning trajectories as positive samples ($y = 1$) and states from baseline losing trajectories as negative samples ($y = 0$). The objective is to minimize the Binary Cross Entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(R_\theta(\phi(s_i)))$ is the network's predicted "quality" of the state. This learned probability is then used directly as the evaluation heuristic in the MCTS simulation step.

### 3.3 Hypotheses

We structure our experiments around three core hypotheses to systematically evaluate the impact of reward shaping and model architecture:

- **H1 (Efficacy of Learned Rewards):** We hypothesize that agents using a learned Neural Network-based reward function (derived from expert gameplay) will achieve a significantly higher win rate against the baseline *MctsAi* than the original TOVOR agent, which relies on a simple hand-crafted HP-difference heuristic. This would validate the utility of Inverse Reinforcement Learning in this domain.
- **H2 (Architecture Depth vs. Speed):** We hypothesize that a deeper neural network (4-layer MLP) will outperform a shallow network (2-layer MLP) despite the computational overhead. We expect that the complex, non-linear relationships in fighting games (e.g., spacing + energy + frame advantage) require higher model capacity to capture effectively, outweighing the benefit of raw search speed provided by the smaller network.
- **H3 (Sparse vs. Dense Shaping):** We hypothesize that **Dense Rewards** (trained on short-term HP gains) will provide a more granular and effective training signal than **Sparse Rewards** (trained on long-term match outcomes). Given the dynamic nature of fighting games, we expect immediate feedback on tactical actions to guide the MCTS search more effectively than a delayed win/loss signal.

### 3.4 Architecture and Training

We designed two distinct neural network architectures to test H2. Both are Multi-Layer Perceptrons (MLPs) implemented in PyTorch. The final architectures for both models are illustrated in Fig. 2.

- **FastNet (Shallow):** A lightweight network with 2 hidden layers ($32 \to 16$ neurons). Designed for maximum inference speed ($< 0.01$ms) to allow high MCTS simulation counts.
- **DeepNet (Deep):** A deeper network with 4 hidden layers ($128 \to 64 \to 32 \to 16$ neurons). Designed to capture complex state interactions.

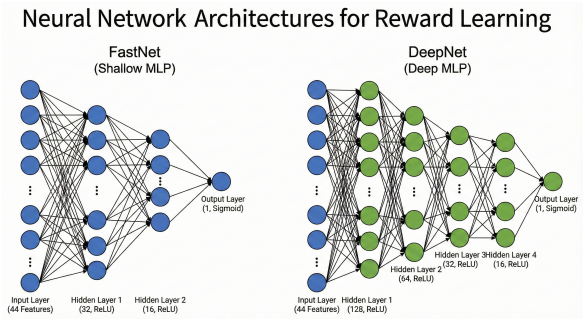


Figure 2: Model architectures used in this work. FastNet is a shallow 2-layer MLP, while DeepNet is a 4-layer MLP designed to capture higher-order interactions between game-state features.

**Training Protocol:** All models were trained on a local machine with a single NVIDIA GeForce RTX 4090 Laptop GPU (16 GB VRAM). We trained four distinct models: **SparseFast**, **SparseDeep**, **DenseFast**, and **DenseDeep**.

- **Loss Function:** Binary Cross Entropy (BCE) Loss, as defined in our IRL formulation.
- **Optimizer:** Adam with a learning rate of 0.001.
- **Batch Size:** 64.
- **Epochs:** 30. We validated convergence by monitoring validation accuracy curves, which plateaued after approximately 10-15 epochs, indicating sufficient training without overfitting.

Table 1: Validation Accuracy and Training Time for Each Model

| Model | Validation Accuracy (%) | Training Time (s) |
|---|---|---|
| SparseFast | 77.65 | 4.18 |
| SparseDeep | 72.04 | 15.74 |
| DenseFast | 64.14 | 3.66 |
| DenseDeep | 67.35 | 15.67 |

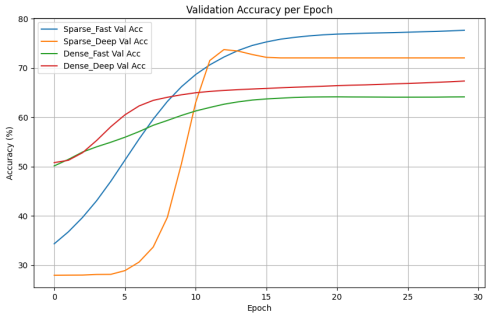


Figure 3: Validation accuracy curves during training for all four models. Sparse models converge faster and reach higher accuracies, while Dense models exhibit noisier learning dynamics.

### 3.5 MCTS Integration

The trained models were integrated into the TOVOR agent's Java codebase. We replaced the standard evaluation function with a forward pass of the neural network. To address the "Event Horizon" problem where agents fail to see long-term consequences (like projectiles), we set the MCTS simulation horizon to **30 frames** (0.5 seconds) and enforced a strict 14ms time budget per frame to prevent freezing.

## 4 Data

### 4.1 Dataset Construction

Our dataset was derived from two sources of FightingICE replays:

1. **Positive Trajectories (Expert):** 100 replays of *BlackMamba* (a competition winner) defeating *MctsAi*.
2. **Negative Trajectories (Non-Expert):** 100 replays of the original *TOVOR* losing to *MctsAi*.

This resulted in a dataset of approximately **700,000 frames**.

### 4.2 Feature Engineering and Preprocessing

We developed a custom Python parser to extract 44 features from the raw JSON frame data.

- **Normalization:** All continuous variables (HP, Energy, Distance, Velocity) were normalized to the range $[-1, 1]$ or $[0, 1]$ using standard scaling.

- **One-Hot Encoding:** Categorical variables such as Character State (Stand, Crouch, Air, Down) and Action Type (Attack, Guard, Move) were one-hot encoded to provide clear signals to the network.
- **Derived Features:** We engineered specific features to capture game dynamics, including "Closing Speed" (relative velocity towards opponent) and "Projectile Threat" (distance to nearest active hazard).

### 4.3 Labeling Strategy (Reward Shaping)

To test H3, we created two sets of labels for the same dataset:

- **Sparse Labels (Strategic):** Labels were assigned based on the file source. All frames from Expert matches were labeled $y = 1$ (Positive), and all frames from Losing matches were labeled $y = 0$ (Negative). This means all frames in a win or loss are labelled the same.
- **Dense Labels (Tactical):** Labels were calculated using an N-step lookahead. For each frame, we calculated the net HP change at $t + 10$, $t + 20$, $t + 30$, and $t + 80$ frames. If the weighted sum of these future HP gains was positive, the frame was labeled $y = 1$. This isolates "good moves" even within losing matches. This allows frames within the same game to be labelled positively or negatively giving more diverse or dense information.

### 4.4 Data Split

The final dataset was shuffled and split into 80% Training and 20% Validation sets to evaluate model generalization.

## 5 Experiments and Results

In this section, we evaluate the performance of our trained agents against established baselines and analyze the trade-offs between model depth, inference speed, and reward density. We selected *MctsAi* as the primary opponent for evaluation, as it represents a robust standard in the FightingICE environment. We also compare our agents against *BlackMamba*, a state-of-the-art script-based agent, and *Tovor*.

The agents trained for this study vary along two axes: architecture depth (Deep vs. Fast) and reward signal (Sparse vs. Dense). The results of these matchups, along with relevant Head-to-Head (H2H) experiments, are summarized in Table 2.

Table 2: Win rates (WR) and Average End-of-Game HP for various agent matchups. P1 represents the agent under test, and P2 represents the opponent.

| Player 1 (P1) | Player 2 (P2) | WR P1 (%) | WR P2 (%) | Avg HP P1 | Avg HP P2 |
|---|---|---|---|---|---|
| *Baselines* | | | | | |
| **BlackMamba** | MctsAi | **100** | 0 | **389** | 0 |
| Tovor | **MctsAi** | 26 | **74** | 119 | **168** |
| *Experimental Agents vs. MctsAi* | | | | | |
| **DenseDeep** | MctsAi | **54** | 46 | **140** | 110 |
| **DenseFast** | MctsAi | **68** | 32 | **195** | 167 |
| **SparseDeep** | MctsAi | **93** | 7 | **210** | 42 |
| SparseFast | **MctsAi** | 15 | **85** | 163 | **253** |
| *Head-to-Head (H2H)* | | | | | |
| **SparseFast** | SparseDeep | **57** | 43 | **47** | 41 |
| **SparseFast** | DenseDeep | **60** | 40 | **78** | 57 |
| SparseFast | **DenseFast** | 20 | **80** | 57 | **114** |
| **SparseDeep** | DenseFast | **93** | 7 | **153** | 43 |
| **SparseDeep** | DenseDeep | **61** | 39 | **57** | 37 |
| **DenseFast** | DenseDeep | **97** | 3 | **232** | 20 |

### 5.1 Performance Against Standard Baselines

The baseline comparisons reveal significant improvements over previous iterations. While *BlackMamba* remains dominant with a 100% win rate against *MctsAi*, three of our experimental models demonstrated the ability to outperform the *Tovor* baseline significantly. *Tovor* achieved only a 26% win rate against *MctsAi*. In contrast, *SparseDeep* achieved a remarkable 93% win rate, approaching the dominance of *BlackMamba* and significantly outperforming its dense-reward counterparts.

### 5.2 Analysis of Learned Behaviors and Failure Modes

A deeper analysis of the agents' action distributions reveals distinct behavioral patterns that explain the performance gaps observed in Table 2.

The gold standard for strategy in this environment is demonstrated by *BlackMamba*. Its core tactic involves aggressive play to corner the opponent. Once the opponent is cornered, *BlackMamba* often repeats the same combo: a crouching kick, linked into a standing kick, linked into a either a throw that can loop into the same combo, or a dragon punch that deals more damage but ends the loop. If *BlackMamba* opts for the throw ender, then the opponent is able to block the subsequent crouching kick to avoid the combo; however, the crouching kick can only be blocked while crouching, so the opponent will still be hit if they block while standing.

Our experimental agents show varying degrees of success in replicating this optimal "cornering" strategy:

- **DenseDeep:** This agent has a straightforward strategy. If the opponent jumps in, it anti-airs the opponent and knocks them down with a dragon punch (which is this agent's most-used move). Next, when the opponent gets up and becomes actionable, this agent waits for them to jump and knocks them down again with another dragon punch. This process loops for a while until this agent decides to do something else or the opponent chooses to not jump when becoming actionable. However, *DenseDeep* usually loses interactions in the neutral game, so it is overly reliant on its advantage state. This explains its decent win rate of 54% against *MctsAi*. As the dense reward function is tied to HP differentials, this agent may struggle in the neutral game because it is trying to get more damage instead of a knockdown (which leads to this agent's win condition).
- **SparseDeep:** This agent has the same win condition of *DenseDeep*. Where it differs is the neutral game: *SparseDeep* frequently uses crouching kick whenever it has a chance of hitting the opponent (in fact, crouching kick is this agent's most used action). Whenever it hits one of these, it combos the crouching kick into a dragon punch which sets up this agent's win condition. With this new combo, *SparseDeep* is able to set up its win condition whether the opponent is in the air or grounded. This resulted in this agent having a 93% win rate against *MctsAi*. It is likely that the sparse reward signal incentivized this agent to prioritize the *state* of winning rather than damage, which is why this agent used crouching kick instead of a more damaging move or a move that's more likely to hit.
- **Playing to One's Win Condition:** A correlation emerges: agents that are better able to set up win conditions perform better. *DenseDeep* does decently well by having a way to transition into its win condition against aerial opponents. *SparseDeep* improves upon this by having another transition that works on grounded opponents. *BlackMamba* does this the best by cornering the opponent (removing their ability to move backwards) and starting their win condition combo before the opponent can do anything except block instead of waiting for the opponent to jump so *BlackMamba* can anti-air.

### 5.3 Head-to-Head Dynamics and Latency

The Head-to-Head (H2H) experiments reveal an interesting, seemingly paradoxical behavior in that no agent had a perfect record. *DenseFast* came close, beating both *SparseFast* and *DenseDeep* by wide margins, but it lost to *SparseDeep* by a similarly wide margin. Meanwhile, *SparseDeep* lost to *SparseFast*.

This Rock-Paper-Scissors dynamic is likely best explained by a difference in playstyles and limited training data. All of the learned reward functions were trained on data from two matchups: *Black-*

*Mamba* vs. *MctsAi* and *Tovor* vs. *MctsAi*. It's probable that *DenseDeep*, with its great capacity for learning, learned how *MctsAi* plays and how to best counter it. This accounts for the 54% win rate *DenseDeep* has against *MctsAi* (which is a huge improvement on the 26% win rate *Tovor* has) and for *DenseDeep* losing all of its head-to-head matches.

Another interesting observation is that *SparseDeep*, the agent that performed the best against *MctsAi* (other than *BlackMamba*), lost to *SparseFast*, the agent that performed the worst against *MctsAi*. Qualitative observations of this match shows that *SparseFast* typically does not jump after becoming actionable from being knocked down. Since the win condition of *SparseDeep* is to anti-air a jump after knocking down their opponent, *SparseDeep* lost its main win condition that helped it beat *MctsAi*. With a wider variety of training data, it's possible that *SparseDeep* would have learned a different win condition that can be applied more generally.

### 5.4 Revisiting Hypotheses

Based on the experimental data, we can now explicitly evaluate our original hypotheses:

- **H1 (Efficacy of Learned Rewards): Supported.** The core premise of the study held true. The agent *SparseDeep*, utilizing a learned reward function, achieved a 93% win rate against *MctsAi*, a massive improvement over the baseline *TOVOR*'s 26%. This validates that Inverse Reinforcement Learning can capture complex, implicit strategies that simple hand-crafted heuristics miss.
- **H2 (Architecture Depth vs. Speed): Partially Supported / Nuanced.** While deep networks were necessary to learn the optimal way to defeat the baseline agent *MctsAi* (as seen in *SparseDeep*'s performance), the "bigger is better" assumption failed against new opponents that used different strategies (namely, *SparseFast*). This suggests a need for less biased training data so that deep networks can learn a more general strategy.
- **H3 (Sparse vs. Dense Shaping): Rejected.** We hypothesized that dense, granular feedback would be superior. The results proved the opposite. Agents trained on dense rewards (*DenseDeep*, *DenseFast*) converged to suboptimal, "greedy" policies that prioritized maximizing damage over setting up a win condition. The sparse signal was critical for allowing the agent to use low-damage actions (like crouching kick) when they could enable advantageous situations. This shows denser rewards can result in converging to suboptimal and greedy behaviors.

## 6 Conclusion

### 6.1 Summary and Explicit Limitations

In this work, we explored the impact of reward sparsity and model depth on Fighting Game AI. We found that sparse rewards combined with deep architectures yield the most strategically robust agents, capable of replicating the ability to set up win conditions like *BlackMamba*. However, there was a critical limitation in our work: the training data was not varied enough. This caused the deep agents to learn tactics that only worked against agents in the training set, setting these agents up to fail when pitted against a novel agent.

### 6.2 Concrete Future Work

Future work should train on a wider variety of models. Because *BlackMamba* only played against *MctsAi*, there was no way for the learned reward function to know what *BlackMamba* would do against a different opponent. A varied training set allows for learned reward functions to capture generally good strategies instead of gimmicky tactics that only work well against specific agents.

Recall that the dense agents in this project did poorly because the learned reward function classified states as being "good" or "bad" during its training based on a simple HP differential. Future research should make a dense agent that classifies states based on a learned reward function (like the one used in *SparseDeep*). This would improve the quality of the training data by having the agent recognize mistakes made by experts (in this case, *BlackMamba*) and good plays made by weak agents (*Tovor*).

| Team Member | Contributions |
|---|---|
| Austin McGregor | Extracted TOVOR agent; integrated heuristics; led data collection |
| Ray Wang | Validated MctsAI23i setup; debugged agent behavior |
| Jake Nardomarino | Extracted BlackMamba model; helped with initial IRL pipeline |
| Agnibho Chattarji | Built Deep IRL system and trained models; engineered features; implemented 4 novel TOVOR agents |
| Everyone | Assisted with data collection |

# References


[1] Steven Adams et al. "A Survey of Inverse Reinforcement Learning". In: *Artif. Intell. Rev.* 56 (2022), pp. 4419–4448.

[2] Jonathan Ho and Stefano Ermon. "Generative adversarial imitation learning". In: *Advances in Neural Information Processing Systems 29*. 2016.

[3] Dae-Wook Kim, Sungyun Park, and Seong-il Yang. "Mastering Fighting Game Using Deep Reinforcement Learning with Self-Play". In: *2020 IEEE Conference on Games (CoG)*. 2020, pp. 576–583. DOI: 10.1109/CoG47356.2020.9231639.

[4] Andrew Y. Ng, Daishi Harada, and Stuart Russell. "Policy invariance under reward transformations: Theory and application to reward shaping". In: *Proceedings of the Sixteenth International Conference on Machine Learning (ICML '99)*. 1999, pp. 278–287.

[5] Inseok Oh et al. "Creating Pro-Level AI for a Real-Time Fighting Game Using Deep Reinforcement Learning". In: *IEEE Transactions on Games* 14.2 (2022), pp. 212–220. DOI: 10.1109/TG.2021.3049539.

[6] Brian D. Ziebart et al. "Maximum entropy inverse reinforcement learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2008.